Phase 2 — Access Boundary Analysis

Date: 2026-02-27 **Analyst**: Claude Code **Scope**: MCP server at `mcp/` directory — 39 registered tools (not 33 as initially estimated), 7 resources, 3 prompts

---

Corrections from Plan Estimates

The plan estimated 33 tools. The actual code registers **39 tools** in `--context all` mode:

- Discovery: 8 tools (plan: ~5)
- UI Read: 9 tools (plan: ~5)
- Workflow Read: 5 tools (matches plan)
- Search: 2 tools (matches plan)
- Workflow Write: 3 tools (matches plan)
- UI Write: 8 tools (plan: ~5)
- Validation: 1 tool (matches plan)
- Analysis: 3 tools (matches plan)

Additionally, `client.go` contains two methods that have **no corresponding MCP tools**: `UpdateForm` (line 309) and `UpdateFormField` (line 319). These are significant unplanned gaps.

Part A — Under-Exposure: What Can Clients NOT Do (But Should)?

1. Missing CRUD Operations

CRUD coverage is analyzed per domain based on code inspection of `mcp/internal/tools/` and `mcp/internal/client/ichor.go`.

Domain	CREATE	READ	UPDATE	DELETE	Coverage
Workflows	✅ `create_workflow`	✅ `list_workflows`, `get_workflow`	✅ `update_workflow`	❌ absent	**75%**
Page Configs	✅ `create_page_config`	✅ `list_pages`, `get_page_config`, `get_page_content`	✅ `update_page_config`, `update_page_content`	❌ absent	**75%**
Forms	✅ `create_form`	✅ `list_forms`, `get_form_definition`	❌ unplanned gap[1]	❌ absent	**50%**
Form Fields	✅ `add_form_field` (append-only)	❌ partial (only via parent form)	❌ unplanned gap[1]	❌ absent	**25%**
Table Configs	✅ `create_table_config`	✅ `list_table_configs`, `get_table_config`	✅ `update_table_config`	❌ absent	**75%**
Page Actions	❌ absent	✅ `get_page_actions`, `get_page_action`	❌ absent	❌ absent	**25%**
Action Templates	❌ absent	✅ `list_action_templates` (listing only, no get-by-ID)	❌ absent	❌ absent	**25%**

[1] **`UpdateForm` and `UpdateFormField` methods exist in `client.go` (lines 309, 319) but no MCP tools call them.** These are unambiguously unplanned gaps — the backend plumbing is present but the tools were never wired.

DELETE is universally absent. No `Delete*` method exists anywhere in `client.go`. This appears intentional — the REST API DELETE endpoints may exist but the client layer was never extended to call them for MCP purposes.

Business impact of 0% DELETE coverage:

- An agent that creates a broken workflow cannot clean it up
- An agent that creates a test page config cannot remove it after testing
- An agent that adds a malformed form field cannot remove it — the form is permanently corrupted until a human intervenes
- This forces a human-in-the-loop for any destructive correction, which may be **intentional design** (see Part C)

---

2. Missing Data-Plane Access

The MCP operates exclusively on the **configuration plane**. Zero access to:

Data Domain	Accessible?	Automation Blocked
Orders / order line items	❌	Cannot query order data before triggering workflows
Inventory (items, locations, zones)	❌	Cannot verify stock levels in workflow configuration
Products / brands / categories	❌	Cannot cross-reference product data when building forms
Users / employees / HR	❌	Cannot look up approvers when designing approval workflows
Customers / contacts	❌	Cannot validate customer data for notification workflows
Assets	❌	Cannot inspect asset state before automation
Workflow execution history	❌	**Cannot verify if a created workflow actually fired**
Workflow execution status	❌	**Cannot see if a running workflow is in-flight**
Form submission data	❌	Cannot see what users actually submitted via forms
Alerts generated by workflows	❌	Cannot check if workflow alert actions succeeded
Audit logs	❌	Cannot review what the agent itself changed

Critical automation blocker: An agent that creates a workflow has no way to verify:

1. That the trigger correctly identifies the entity type
2. That the workflow fired when test data was created
3. That the workflow execution succeeded or failed
4. Whether a previous duplicate workflow is already handling the same trigger

This forces a human to manually verify every workflow the agent creates — there is no self-verification loop available.

What OpenClaw clients CANNOT automate as a result:

- "Check if orders over $5000 are being escalated" → no execution history
- "Fix the workflow that's been failing for the last hour" → no error logs
- "Show me all workflows triggered by this order" → no execution records
- "Build a workflow, then verify it worked on a test order" → no data-plane feedback

---

3. Missing Configuration Operations

Operation	Available?	Notes
Manage user accounts	❌	No users endpoint in client at all
Manage roles	❌	No roles endpoint in client
Manage table-level permissions	❌	No permissions endpoint in client

Operation	Available?	Notes
System settings / environment config	❌	No settings endpoint
Alert configuration (alert types, channels)	❌	No alert config in client
Workflow activation toggle	❌	`is_active` exists in model but no dedicated toggle tool

Note on workflow activation: There is no `activate_workflow` or `deactivate_workflow` tool. An agent wishing to deactivate a workflow must use `update_workflow` and pass the full workflow payload with `is_active: false` — which requires knowing the full current state first. There is no lightweight toggle.

---

4. Operational Gaps

Operation	Available?	Impact
Manually trigger a workflow for testing	❌	No way to test a newly-created workflow without creating real data
Pause a running workflow execution	❌	No way to halt a runaway workflow mid-execution
Cancel a pending approval in a workflow	❌	No way to unblock stuck human-approval steps
View workflow execution history	❌	No observability into whether automation is working
View workflow execution logs per run	❌	No debugging capability
Manage workflow execution schedules	❌	Temporal-based scheduling is invisible to MCP

---

5. Integration Gaps

Gap	Available?	Impact
Batch create/update tools	❌	Every item must be created individually; building 20-field forms is 20+ API calls
Export configuration as JSON/YAML	❌	No way to snapshot or backup a configuration set
Import configuration from file	❌	No way to apply a pre-built configuration bundle
Register webhook callbacks	❌	No way for agent to receive async results
Copy/clone a workflow	❌	No duplication tool; agents must recreate from scratch

---

6. Gap Prioritization: Enterprise Impact × Implementation Cost

Gap	Enterprise Impact	Implementation Cost	Priority
`update_form` / `update_form_field` tools (client methods exist!)	HIGH — forms are unusable after creation without update	**LOW** — client methods exist, just need tool wiring (~20 LOC each)	**P0**
Workflow execution history read	HIGH — no observability into automation	MEDIUM — new read-only endpoint needed	**P1**
DELETE operations	HIGH — can't clean up broken configs	MEDIUM — REST API DELETEs likely exist	**P1**
Page action write tools	MEDIUM — agents can read but not build page actions	MEDIUM — new client methods + tools	**P2**
Workflow activation toggle	MEDIUM — clunky update-full workaround	LOW — thin wrapper over update_workflow	**P2**

Gap	Enterprise Impact	Implementation Cost	Priority
Data-plane read access	MEDIUM — limits self-verification loops	HIGH — requires scope decisions on what data agents should see	**P3**
Action template write tools	LOW — templates are shared infrastructure	LOW — straightforward CRUD	**P3**
Batch operations	LOW — verbose but functional	HIGH — significant complexity	**P4**

Part B — Over-Exposure: What Can Clients Do (But Shouldn't, Or Not By Default)?

1. Full Schema Introspection (`search_database_schema`)

Evidence from code (`mcp/internal/tools/search.go:14–62`):

```
// No args: list all schemas.
data, err := c.GetSchemas(ctx)
// Schema only: list tables.
data, err := c.GetTables(ctx, args.Schema)
// Schema + table: get columns AND relationships.
columns, err := c.GetColumns(ctx, args.Schema, args.Table)
relationships, err := c.GetRelationships(ctx, args.Schema, args.Table)
```

Three calls. No args. The entire database structure is enumerable.

What this exposes to any authenticated agent:

- All PostgreSQL schema names (core, hr, geography, assets, inventory, products, procurement, sales, config, workflow)
- All table names within each schema
- Every column name, data type, nullable flag, and default value
- Every foreign key relationship between tables (exact column references, cascade rules)

Who could be harmed:

- Any enterprise agent with a stolen or leaked Bearer token
- A prompt-injected agent (malicious instructions in a workflow name, form label, or product description that the agent reads and acts on)
- An over-privileged agent that was intended only for workflow building

Risk: The backend endpoint hits `auth.RuleAdminOnly`, but enterprise agents are typically issued admin-level tokens to do their job. The auth gate is real but ineffective against the realistic threat model.

Recommended control: Add a `restricted_schemas` option to the tool, defaulting to only `config` and `workflow` schemas. Block `core` (users), `sales` (orders/customers), `hr` (employees) unless the caller explicitly requests broader access. Gate broad schema access (`core.*` , `sales.*`) behind an admin-only check that also verifies the token has an explicit `schema_introspect` claim.

---

2. Workflow Creation Without Human Review

Evidence from code (`mcp/internal/tools/write_workflow.go:55–83`):

```
// shouldValidate is true by default — but validates graph structure only,
// NOT semantic/business safety.
if shouldValidate {
    // Only catches: malformed graphs, missing required fields, cycle violations.
    // Does NOT catch: "send 10,000 emails", "allocate all inventory",
```

```
        //                  "create alert for every single row update"
    }
    // If structural validation passes, the workflow is immediately LIVE.
    data, err := c.CreateWorkflow(ctx, args.Workflow)
```

The validate-first pattern only catches what the graph validator checks: DAG validity, required fields, edge consistency. It cannot catch semantically dangerous configurations such as:

- A workflow that sends email notifications triggered on `on_update` for a high-frequency entity like order line items — potentially thousands of emails per minute
- A workflow that calls `allocate_inventory` for every product creation, exhausting stock
- A workflow that triggers `seek_approval` from an external email address controlled by an attacker

Who is harmed: All users, downstream systems, and data integrity. A prompt-injected agent can construct a valid workflow (passes structural validation) that causes real operational damage.

Recommended control: Add a `mode: "draft"` option to `create_workflow` that saves the workflow in an inactive/pending state requiring human activation. Gate the `is_active: true` creation behind a separate step that requires human confirmation, or restrict it to `auth.RuleAdminOnly`.

---

3. `validate=false` Bypass on Workflow Writes

Evidence from code (`mcp/internal/tools/write_workflow.go:56`):

```
shouldValidate := args.Validate == nil || *args.Validate
```

And in the tool description (line 46):

```
"Set validate=false to skip pre-validation."
```

This bypass is **documented and fully exposed** to any authenticated agent. It is intended as a developer escape hatch but has no access control. Any enterprise client agent can call:

```
{"workflow": {...}, "validate": false}
```

...and bypass all structural validation.

Scenarios where this is exploitable:

- An agent that received prompt-injected instructions to "deploy immediately without validation"
- A buggy LLM that produces malformed workflows and passes `validate: false` to work around repeated errors (LLMs do this)
- A legitimate agent that genuinely needs to skip validation but should not have that capability

Who is harmed: System integrity. Unvalidated workflows can create broken graph states that cause the Temporal worker to fail at runtime, potentially blocking legitimate workflow executions.

Recommended control: Remove `validate=false` from the tool's input schema entirely, or gate it behind `auth.RuleAdminOnly` check at the REST API level. The MCP server currently has zero ability to restrict per-argument access — this requires a backend change.

---

4. Enum Introspection as Business Vocabulary Harvesting

Evidence from code (`mcp/internal/tools/search.go:64-91`):

```
// No special auth check at tool level.
// Backend endpoint: GET /v1/introspection/enums/{schema}
// Backend auth: auth.RuleAny (any authenticated user)
data, err := c.GetEnumOptions(ctx, args.Schema, args.Name)
```

`GetEnumOptions` calls `/v1/config/enums/{schema}/{name}/options` — note this is the **config** endpoint with human-readable labels, not the raw introspection endpoint.

What this exposes:

- All PostgreSQL enum type names per schema (inventory status codes, asset conditions, order states, approval statuses, etc.)
- Human-readable labels for every enum value — the exact business vocabulary used in the UI
- This is gated by `auth.RuleAny` — the lowest possible auth bar

Risk context: An attacker with any valid token (even a read-only user token) can enumerate all business status transitions, approval workflow states, and condition codes. Combined with `search_database_schema`, this provides a detailed map of the business domain model.

Recommended control: Elevate enum introspection to `auth.RuleAuthenticated` with table-level permission checks (same as other config reads), or restrict it to the `workflow` and `config` schemas only.

---

5. Context Mode `all` as Default

Evidence from code (`mcp/cmd/ichor-mcp/main.go:29`):

```
contextMode := flag.String("context", "all", "Tool context filter: all, workflow, or tables")
```

The default invocation (`ichor-mcp --api-url ... --token ...`) registers all 39 tools simultaneously.

An agent scoped to "analyze and improve approval workflows" holds in its context:

- `create_table_config` — can create UI widget configurations
- `create_form` / `add_form_field` — can build data entry forms
- `create_page_config` / `create_page_content` — can create entirely new pages
- `update_page_config` — can modify existing page layouts

These tools are completely irrelevant to workflow analysis but are present and callable.

Principle of least privilege violation: The tool count for a workflow-only agent should be ~17 (the `workflow` context), not 39. Every extra tool is additional attack surface and additional LLM decision surface — in non-frontier models, this measurably degrades tool selection accuracy.

Recommended control: Invert the default — make `--context workflow` the default, or at minimum make `--context all` require an explicit `--allow-all-contexts` flag. Document which personas use which context modes.

---

6. Prompts Pre-fetch Sensitive Context Without Scope Checks

Evidence from code (`mcp/internal/prompts/prompts.go`):

The `build-workflow` prompt (line 26-27):

```
actionTypes, _ := c.GetActionTypes(ctx)
triggerTypes, _ := c.GetTriggerTypes(ctx)
```

This injects the **complete action type catalog** — including destructive action types (inventory allocation, external webhooks, email notifications, seeks-approval with external parties) — into the agent's context before the agent has stated anything about the intended workflow.

The `design-form` prompt (line 142-145) explicitly instructs the agent to:

```
"First, explore the entity's database schema using search_database_schema to understand its
columns and relationships"
```

This hardcodes a call to `search_database_schema` as a first step of any form design session, ensuring full schema introspection happens automatically on every form design invocation.

Risk: Front-loading destructive capability knowledge into every prompt invocation widens the prompt injection attack surface. An adversary whose text appears anywhere in the "entity" argument or early in the agent's context gains awareness of all available action types and their configuration schemas.

Recommended control:

- `build-workflow`: Filter pre-fetched action types by the declared category (notification, approval, inventory, etc.) rather than dumping all 17 types
- `design-form`: Remove the `search_database_schema` instruction from the hardcoded prompt text; let the agent decide if it needs schema info

---

7. No Differentiation Between Read-Only and Read-Write Tokens

Evidence from code: `client.go` uses a single Bearer token for all requests. `main.go` accepts one `--token` flag.

The problem:

- A token issued to a reporting/analysis agent has identical capabilities to a configuration deployment agent
- There is no way to issue a token that allows `list_workflows` + `get_workflow` + `analyze_workflow` but blocks `create_workflow` + `update_workflow`
- The only available scoping mechanism is `--context` mode selection, but `workflow` context still includes all write tools

Consequence: An enterprise deploying an analysis agent must trust that the agent will not autonomously invoke write tools. There is no technical enforcement of this trust boundary.

Recommended control: The MCP server could accept a `--read-only` flag that filters out all write tools at registration time (tools tagged with a write annotation), providing a safe read-only context without requiring backend changes. This is a purely MCP-layer mitigation that can be implemented without touching the REST API.

Part C — Intentional vs. Accidental Restrictions

Restriction	Classification	Evidence
DELETE absent for all domains	**By design**	No `Delete*` method in `client.go` — consistently absent, not just missing wiring
User/role/permission management absent	**By design**	No auth management in client at all — the MCP is scoped to configuration, not auth infrastructure

Restriction	Classification	Evidence
Business data (orders, inventory, etc.) absent	**By design**	The MCP's stated scope is configuration-plane; data-plane access would require a separate design decision
`update_form` tool missing	**Unplanned**	`UpdateForm` at `client.go:309` exists and is never called; this is a wiring oversight
`update_form_field` tool missing	**Unplanned**	`UpdateFormField` at `client.go:319` exists and is never called; wiring oversight
Page action write tools absent	**Ambiguous**	No client methods exist either — could be deliberate (stable config) or an oversight
Action template write tools absent	**Ambiguous**	No client methods; templates may be managed outside of agent scope
Workflow execution history absent	**Ambiguous**	Could be deliberate (Temporal is a separate concern) or planned for future
Workflow activation toggle absent	**Unplanned**	The model has `is_active` and the REST API likely supports PATCH; no dedicated toggle was built
Action template read limited to listing (no get-by-ID)	**Unplanned**	Other entities have both list and get-by-ID; this inconsistency looks accidental

Summary for OpenClaw Clients

"By design" gaps to accept in your integration strategy:

- Plan for all cleanup operations to require human intervention (no DELETE)
- Do not rely on MCP for user/permission administration
- Do not rely on MCP to query business data; use the REST API directly for data-plane queries

"Unplanned" gaps to request on the roadmap:

- `update_form` and `update_form_field` tools (1-day fix, client methods exist)
- `delete_workflow` / `delete_page_config` / `delete_form` / `delete_table_config` (medium effort)
- Workflow execution history read endpoint + MCP tool (medium-high effort)
- Consistent get-by-ID for action templates (trivial)

Depth Ratings: Gap Business Impact and Remediation Path

Gap	Business Impact Statement	Remediation Path
`update_form` / `update_form_field` missing	Once a form is created, field labels, validation rules, and config cannot be changed by an agent. Any form correction requires human API access. This makes agent-driven form iteration impossible.	Wire existing `client.UpdateForm()` and `client.UpdateFormField()` to new MCP tools. ~40 LOC total.
No DELETE operations	Agents cannot clean up test artifacts, broken configurations, or deprecated workflows. Tenant deployments accumulate stale configs permanently.	Add `Delete*` methods to `client.go`, create `delete_workflow`, `delete_page_config`, `delete_form`, `delete_table_config` tools with explicit `confirm: true` required field as a human-intent signal.

Gap	Business Impact Statement	Remediation Path
No workflow execution history	Agents have no observability loop: create workflow → cannot verify it ran → cannot diagnose failures. This eliminates any self-correcting agent behavior.	Add read-only execution history endpoint to REST API, add `list_workflow_executions` and `get_execution_result` MCP tools.
No page action write tools	Agents can build complete pages with content blocks but cannot add buttons or navigation actions. Pages built by agents are non-interactive.	Add `CreatePageAction`, `UpdatePageAction`, `DeletePageAction` to `client.go`, wire MCP tools.

Over-Exposure Risk Summary

Risk	Who is Harmed if Exploited	Recommended Control
Full schema introspection	All data — complete DB model map enables targeted exploitation	Restrict to `config` / `workflow` schemas by default; gate `core` / `sales` behind explicit admin request
Workflow creation without human review	System integrity and operational safety — destructive workflows go live immediately	Add `mode: "draft"` to `create_workflow`; require explicit activation step
`validate=false` bypass	Graph integrity — unvalidated workflows can corrupt execution state	Remove parameter from tool schema entirely; restrict to admin API flag only
Enum introspection at `auth.RuleAny`	Business vocabulary exposure to any token holder	Elevate to table-level permission check or restrict to `config` / `workflow` schemas
`--context all` default	Attack surface — 39 tools for any agent, regardless of intended use	Invert default; require explicit `--context all` opt-in
Prompts pre-load destructive context	Widened prompt injection surface — destructive capability knowledge in every prompt call	Filter action types by declared use-case in `build-workflow` prompt
No read-only token scoping	Write capabilities exposed to read-only agents — no technical enforcement	Add `--read-only` flag to MCP server filtering write tools at registration time